

Organização e Arquitetura de Computadores

Jorge Augusto Salgado Salhani

no. USP: 8927418

Junho, 2026

1 Lista 4

1. Uma das características das arquiteturas RISC é que a maioria das instruções possuem tamanhos iguais. Quais as consequências e vantagens desta características? Existe alguma desvantagem?

[SOLUÇÃO]

Ao utilizar instruções de tamanho fixo, é mais fácil para arquiteturas RISC utilizar pipeline. A menor complexidade de hardware também permite maior número de registradores, utilizando assim menos tempo para carregar e armazenar valores em memória

Como desvantagens, pelo conjunto limitado de bits alguns campos são mantidos fixos para otimizar a organização (implementação) da RISC, como endereços de registradores origem e destino. Para isso algumas instruções precisam ser armazenadas de forma não intuitiva, como o valor imediato, gerando a necessidade de módulos de extensão e mais sinais de controle. Outra desvantagem consiste no uso não otimizado de instruções, que quando menores que o tamanho fixo, ocupam bits de forma desnecessária.

2. Defina pipeline e sua importância no desempenho dos processadores.

[SOLUÇÃO]

A pipeline é uma solução de organização e arquitetura baseada em uma lógica de linha de montagem e paralelismo.

O processamento de uma instrução ocorre em diversas etapas, por exemplo carregamento (*fetch*), decodificação (*decode*) e execução (*execute*). Como cada etapa pode ser entendida como e implementada para ser independente das demais, enquanto uma primeira instrução está em sua fase de decodificação, a próxima pode ser carregada, e assim sucessivamente.

A implementação de pipeline aumenta a complexidade do processador, mas aumenta consideravelmente seu desempenho, reduzindo o tempo de processamento em alguns casos para 1/3 ou até menos da metade do tempo gasto quando comparados a monociclos (sem implementação de pipeline).

3. Os 5 estágios de dois processadores (a e b) têm as seguintes latências:

	Fetch	Decode	Execute	Memory	Write back
a	300 ps	400 ps	250 ps	500 ps	100 ps
b	200 ps	150 ps	120 ps	190 ps	140 ps

Assuma que, na versão pipeline, cada estágio consome outros 20 ps com os registradores entre os estágios. Para as duas arquiteturas:

- Na versão não pipeline (monociclo), qual o tempo de ciclo? Qual a latência de uma instrução? Qual o *throughput*?
- Na versão pipeline, qual o tempo de ciclo? Qual a latência de uma instrução? Qual o *throughput*?
- Se um dos ciclos pudesse ser dividido ao meio, qual você escolheria? Qual o novo tempo de ciclo? Qual a nova latência? Qual o novo *throughput*?

Consideramos o tempo de ciclo como o tempo para concluir a operação mais demorada. Nesse caso,

- ciclo de 1550 ps
- ciclo de 800 ps

Consideramos latência como o tempo necessário para a finalização de uma tarefa. Assim, como uma instrução deve passar por todas as etapas (caso mais lento, como instruções de **lw**), para a latência

- 1550 ps por tarefa
- 800 ps por tarefa

Consideramos *throughput* como o número de execuções que ocorrem no tempo de um segundo (tarefas por segundo). Assim, considerando apenas instruções mais lentas (pior cenário), para o *throughput* temos

- 1 instrução por 1550 ps, portanto $1/(1550 \times 10^{-12}) \approx 6 \times 10^{-4} \times 10^{12} = 6 \times 10^8$ instruções
- 1 instrução por 800 ps, portanto $1/(800 \times 10^{-12}) \approx 1 \times 10^{-3} \times 10^{12} = 1 \times 10^9$ instruções

Na versão pipeline, cada estágio consome 20 ps, e portanto temos 100 ps adicionais para cada arquitetura. Neste cenário, considerando o ciclo como a etapa de maior duração, temos

- a) ciclo de 520 ps
- b) ciclo de 210 ps

Para a latência (também considerando pior cenário) vale notar que cada etapa demora o mesmo tempo para execução (dado pelo maior tempo). Logo

- a) $5 \times 520 = 2600$ ps por tarefa
- b) $5 \times 210 = 1500$ ps por tarefa

Para o *throughput*, temos 1 instrução por ciclo, mais 4 ciclos iniciais (= **a**) 2080 ps, **b**) 840 ps), até o preenchimento da pipeline. Assim, temos

- a) 1 instrução por 520 ps, logo $1/(520 \times 10^{-12}) \approx 2 \times 10^{-3} \times 10^{12} = 2 \times 10^9$ instruções por segundo
- b) 1 instrução por 210 ps, logo $1/(210 \times 10^{-12}) \approx 5 \times 10^{-3} \times 10^{12} = 5 \times 10^9$ instruções por segundo

4. Considere um pipeline de 5 estágios (IF, ID, EX, MEM, WB). Assuma a seguinte distribuição de instruções que são executadas em um processador:

- 50% são aritméticas/lógicas
- 25% são beq
- 15% são lw
- 10% são sw

Assumindo que o pipeline não vai parar por dependências (dados ou controle), qual a taxa de utilização da memória de dados? Qual a taxa de utilização da porta de escrita no banco de registradores?

[SOLUÇÃO]

Como as únicas instruções que operam com memória de dados são **sw** (para escrita) e **lw** (para leitura), temos 25% de utilização de memória para a distribuição de instruções.

Para o caso seguinte, se considerarmos que operam sobre escrita em registradores instruções do tipo **sw** e também que operações aritméticas resultarão em registradores (**add**, **sub**, ...), temos 65%.

5. Faça a simulação da sequência de código abaixo em um pipeline de 5 estágios e mostre a situação final dos registradores do pipeline (IF/ID, ID/EX, EX/MEM, MEM/WB), considerando também os sinais de controle, ao final dos ciclos de clock 4 e 7. Considere que a primeira instrução está na posição 0 da memória e que o PC começa com 0. Considere também que t1 e t2 não possuem o mesmo conteúdo quando o código é executado.

```

    beq t1, t2, 4
    lw s0, 0(t0)
    add s3, s1, s2
    sw s4, 0(t0)
    addi t1, t1, -4

```

[SOLUÇÃO]

Para 5 estágios, temos os seguintes sinais

Sinal	IF/ID	ID/EX	EX/MEM	MEM/WB
Dado	PC IR	PC	ALUout: resp. ALU	
		A: rs1	B	ALUout
		B: rs2	rd	LMD: Load Mem. Data
		Imm	branchTarget	rd
		rd	zero	
Controle	-	ALUsrc: bco. registr. ou imediato?		
		ALUop	MemRead	
		MemRead: lê da mem.?	MemWrite	
		MemWrite: escreve na mem.?	branch	MemToReg
		branch: é tipo B?	MemToReg	RegWrite
		MemToReg: da ALU ou Mem.?	RegWrite	
		RegWrite: escreve em bco. reg.?		

6. Considere o seguinte trecho de código

- Quais são as dependências de dados verdadeiras? E os *hazards*? Considere que o pipeline tem 5 estágios.
- Assuma um pipeline RISC-V de 5 estágios sem *forwarding*, e que cada estágio demora 1 ciclo. Ao invés de inserir operações *nops*, você indica o processador parar quando se tem *hazards*. Quantos ciclos o processador para? Qual o tamanho de cada parada, em ciclos? Qual o tempo de execução (em ciclos) do programa inteiro?
- Assuma um pipeline RISC-C de 5 estágios com *forwarding* total. Escreva o programa com *nops* para eliminar os *hazards*.

Dependências verdadeiras (RAW - Read After Write) ocorrem quando uma instrução precisa de dados que são resultados de instruções anteriores. Nesse caso temos

- $l1 = l1(l0)$, por $t2$

sinal de dados	cc1	cc2	cc3	cc4	cc5	cc6	cc7
IF/ID.PC	0	4	8	12	16	-	-
IF/ID.IR	beq	lw	add	sw	addi	-	-
ID/EX.PC	-	0	4	8	12	16	-
ID/EX.A	-	t2	(t0)	s1	(t0)	t1	-
ID/EX.B	-	t1	-	s2	s4	-	-
ID/EX.imm	-	4	0	-	0	-4	-
ID/EX.rd	-	-	s0	s3	-	t1	-
EX/MEM.ALUout	-	-	$t2 - t1 \neq 0$	$\text{end}(t0) + 0$	s1+s2	$\text{end}(t0)+0$	$t1+(-4)$
EX/MEM.B	-	-	-	-	s2	s4	-
EX/MEM.rd	-	-	-	s0	s3	-	t1
EX/MEM.brcTgt	-	-	4	-	-	-	-
MEM/WB.ALUout	-	-	-	$t2 - t1 \neq 0$	$\text{end}(t0)+0$	s1+s2	$\text{end}(t0)+0$
MEM/WB.LMD	-	-	-	-	0(t0)	-	-
MEM/WB.rd	-	-	-	-	s0	s3	-

l0: div t2, t5, t8

l1: sub t9, t2, t7

l2: sll t5, a0, 2

l3: mul a1, t9, t5

l4: beq t2, a1, 100

l5: or t8, t5, t2

- l3 = l3(l1, l2), por t9, t5

- l4 = l4(l0, l3), por t2, a1

- l5 = l5(l2, l4), por t5, t2

Hazards ocorrem quando uma instrução tenta ler um registrador antes que o resultado desejado nele tenha sido escrito de fato. Neste caso temos

- l0 / l1, por t2 (hazard, 1 ciclo)

- l1 / l3, por t9 (hazard, 2 ciclos)

- l2 / l3, por t5 (hazard, 1 ciclo)

- l4 / l3, por a1 (hazard, 1 ciclo)

- l4 / l0, por t2 (sem hazard, 4 ciclos)

- l5 / l2, por t5 (sem hazard, 3 ciclos)
- l5 / l4, por t2 (hazard, 1 ciclo)

Considerando a etapa de WB com escrita na primeira metade e leitura na segunda, temos

CC	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
l0	IF	ID	EX	ME	WB	-	-	-	-	-	-	-	-	-	-	-
l1	-	IF	ID _{t2}	ID _{t2}	ID _{t2}	EX	ME	WB	-	-	-	-	-	-	-	-
l2	-	-	-	-	IF	ID	EX	ME	WB	-	-	-	-	-	-	-
l3	-	-	-	-	-	-IF	ID _{t5}	ID _{t5}	ID _{t5}	EX	ME	WB	-	-	-	-
l4	-	-	-	-	-	-	-	-	IF	ID _{a1}	ID _{a1}	ID _{a1}	EX	ME	WB	-
l5	-	-	-	-	-	-	-	-	-	-	-	IF	ID	EX	ME	WB

Incremento de 2 ciclos entre l0 e l1; 2 ciclos entre l2 e l3; 2 ciclos entre l3 e l4. Neste caso temos um total de 16 ciclos.

Ao utilizar forwarding, o resultado de operações da ALU, em estágio de EX já pode ser transferido às instruções seguintes.

- l0: div - em EX já temos resultado de t2, transferido para ID de l1 na virada de bit. Hazard corrigido entre l0 / l1.
- l1: sub - em EX já temos t9. Hazard corrigido entre l1 / l3.
- l2: sll - em EX já temos t5. Hazard corrigido entre l2 / l3
- l3: mul - em Ex já temos a1. Hazard corrigido entre l3 / l4

Com forwarding, todos os hazards são corrigidos, portando o programa leva $N + 4 = 10$ ciclos (N: número de instruções)